

移动端知识笔记系统架构与 UI 设计规范

文档类型：交互与工程落地技术方案书

技术栈：Kotlin + Jetpack Compose (MVI/MVVM 架构)

核心优化：将一维平铺布局升级为“全状态聚焦卡片 + 局部多级折叠分支”结构

一、总体设计哲学与人类直觉推导

在复习高等数学、物理等公式与图表极其密集的学科时，知识的构建通常遵循“大概念 → 具体题型/方法 → 分支情况讨论 → 核心公式/图例”的逻辑。原有一维平铺界面因缺乏边界感，导致用户认知负荷过重。本方案基于以下三条人类直觉进行视觉重构：

- 物理容器直觉（卡片化）**：将一个独立的方法或解题步骤打包进一个纯白色的卡片容器中。利用阴影和卡片物理边界，让大脑在滑动时自动建立记忆区块。
- 行为聚焦直觉（状态激活动态边框）**：手机窄屏下，全局工具栏只能位于底部。通过“点击卡片 → 卡片边框变蓝高亮 → 激活底部工具栏”的逻辑，确保每一次追加块的操作都有明确的目的地。
- 纵向衍生直觉（拒绝画中画套娃）**：卡片内部绝不嵌套卡片，防止横向空间被无限压缩。对于“分情况讨论”的分支逻辑，采用“灰底色面板 + 左侧大缩进 + 动画折叠”的方式呈现，保证公式拥有足够的横向延展空间。

二、数据模型设计 (Data Structure)

为了让 Jetpack Compose 的声明式 UI 能够精准感知到块的类型、嵌套层级以及展开状态，我们需要使用密封类（Sealed Class）以及高度响应式的状态模型。

1. 实体模型定义

```
import java.util.UUID

// 知识点内容块基类
sealed class NoteBlock {
    abstract val id: String

    // 基础文本块
    data class Text(
        override val id: String = UUID.randomUUID().toString(),
        var content: String = ""
    ) : NoteBlock()

    // 图片块（保存本地路径或服务器URL）
    data class Image(
        override val id: String = UUID.randomUUID().toString(),
        val imageUrl: String
    ) : NoteBlock()
```

```

    ) : NoteBlock()

    // LaTeX/数学公式块
    data class Formula(
        override val id: String = UUID.randomUUID().toString(),
        var formulaText: String = ""
    ) : NoteBlock()

    // 局部折叠块（可容纳子基础块，实现分支逻辑）
    data class Toggle(
        override val id: String = UUID.randomUUID().toString(),
        var title: String = "",
        val childBlocks: MutableList<NoteBlock> = mutableListOf(),
        var isExpanded: Boolean = false
    ) : NoteBlock()
}

// 核心卡片模型（一个小点）
data class KnowledgeCard(
    val id: String = UUID.randomUUID().toString(),
    var title: String = "",
    val blocks: MutableList<NoteBlock> = mutableListOf()
)

```

三、交互状态流转与界面布局

编辑状态下的页面流转必须极其清晰。核心交互流程如下：

用户操作行为	界面视觉响应	底层状态改变
点击空白区域 / 刚进入页面	全网卡片均呈淡灰色细边框，底部工具栏置灰/弱化提示	<code>activeCardId = null</code>
点击卡片 B 的任意区域	卡片 B 边框平滑变粗并亮起科技蓝，底部工具栏瞬间激活	<code>activeCardId = cardB.id</code>
点击底部工具栏的【公式】	在卡片 B 内部的最后一行动态冒出一个公式输入框，自动聚焦	<code>cardB.blocks.add(Formula())</code>
点击最下方的【+ 新建小点】	页面底部平滑滚入一张全新空白卡片，且其直接呈蓝色聚焦态	<code>cards.add(newCard)</code> <code>activeCardId = newCard.id</code>

四、Jetpack Compose 核心组件落地实现

以下是经过工程化精简的核心 UI 组件实现代码，包含动效管理、边框过渡以及高级缩进处理。

1. 页面骨架与底部工具栏联动

```
@Composable
fun KnowledgeEditScreen(modifier: Modifier = Modifier) {
    val cards = remember { mutableStateListOf<KnowledgeCard>() }
    var activeCardId by remember { mutableStateOf<String?>(null) }

    Scaffold(
        bottomBar = {
            // 底部动态工具栏, 依赖 activeCardId 是否为空
            BottomActionToolbar(
                isCardActive = activeCardId != null,
                onAddBlockRequested = { blockType ->
                    activeCardId?.let { id ->
                        val targetCard = cards.find { it.id == id }
                        targetCard?.let { card ->
                            val newBlock = when (blockType) {
                                "text" -> NoteBlock.Text()
                                "image" -> NoteBlock.Image(imageUrl = "placeholder")
                                "formula" -> NoteBlock.Formula()
                                else -> NoteBlock.Text()
                            }
                            card.blocks.add(newBlock)
                        }
                    }
                }
            )
        }
    ) { innerPadding ->
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .background(Color(0xFFF5F7FA)), // 浅灰冷色调背景
            verticalArrangement = Arrangement.spacedBy(16.dp),
            contentPadding = PaddingValues(16.dp)
        ) {
            items(cards, key = { it.id }) { card ->
                KnowledgeCardItem(
                    card = card,
                    isActive = card.id == activeCardId,
                    onFocusRequested = { activeCardId = card.id },
                    onDeleteRequested = {
                        if (activeCardId == card.id) activeCardId = null
                        cards.remove(card)
                    }
                )
            }
        }
    }
}
```

```

        // 尾部全局显式新建卡片按钮
        item {
            OutlinedButton(
                onClick = {
                    val newCard = KnowledgeCard()
                    cards.add(newCard)
                    activeCardId = newCard.id // 立即将焦点移交给新卡片
                },
                modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp),
                shape = RoundedCornerShape(12.dp)
            ) {
                Icon(Icons.Default.Add, contentDescription = null)
                Spacer(modifier = Modifier.width(8.dp))
                Text("添加新的知识小点 (新方法/新卡片)")
            }
        }
    }
}
}
}

```

2. 高亮卡片外壳组件

```

@Composable
fun KnowledgeCardItem(
    card: KnowledgeCard,
    isActive: Boolean,
    onFocusRequested: () -> Unit,
    onDeleteRequested: () -> Unit
) {
    // 聚焦态平滑动画过渡
    val borderThickness by animateDpAsState(targetValue = if (isActive) 2.dp else 1.dp)
    val borderColor by animateColorAsState(targetValue = if (isActive) Color(0xFF3B82F6)
else Color(0xFFE5E7EB))
    val cardElevation by animateDpAsState(targetValue = if (isActive) 6.dp else 1.dp)

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable { onFocusRequested() }
            .border(borderThickness, borderColor, RoundedCornerShape(12.dp)),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(containerColor = Color.White),
        elevation = CardDefaults.cardElevation(defaultElevation = cardElevation)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            // Card Header
            Row(verticalAlignment = Alignment.CenterVertically) {
                BasicTextField(
                    value = card.title,

```

```

        onChange = { card.title = it },
        textStyle = TextStyle(fontSize = 16.sp, fontWeight = FontWeight.Bold),
        decorationBox = { innerTextField ->
            if (card.title.isEmpty()) Text("输入小点名称(如:情况一)...", color =
Color.LightGray)
            innerTextField()
        },
        modifier = Modifier.weight(1f)
    )
    IconButton(onClick = onDeleteRequested) {
        Icon(Icons.Default.Delete, contentDescription = "删除整个卡片", tint =
Color(0xFFEF4444))
    }
}

HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color =
Color(0xFFF1F5F9))

// Card Body (循环渲染内部一维块与折叠分支块)
card.blocks.forEach { block ->
    when (block) {
        is NoteBlock.Text -> TextBlockComponent(block)
        is NoteBlock.Image -> ImageBlockComponent(block)
        is NoteBlock.Formula -> FormulaBlockComponent(block)
        is NoteBlock.Toggle -> ToggleBranchComponent(block)
    }
    Spacer(modifier = Modifier.height(8.dp)) // 块内部保持紧凑的 8.dp 间距
}
}
}
}

```

3. 分支折叠组件 (ToggleBranchComponent)

```

@Composable
fun ToggleBranchComponent(block: NoteBlock.Toggle) {
    var isExpanded by remember { mutableStateOf(block.isExpanded) }
    val arrowRotation by animateFloatAsState(targetValue = if (isExpanded) 90f else 0f)

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .background(Color(0xFFF1F5F9), RoundedCornerShape(8.dp)) // 深一些的灰色底色，拉开
视觉差
            .padding(8.dp)
    ) {
        // 折叠触发表头
        Row(

```

```

        modifier = Modifier
            .fillMaxWidth()
            .clickable {
                isExpanded = !isExpanded
                block.isExpanded = isExpanded
            },
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = Icons.Default.KeyboardArrowRight,
            contentDescription = null,
            modifier = Modifier.rotate(arrowRotation),
            tint = Color(#64748b)
        )
        BasicTextField(
            value = block.title,
            onValueChange = { block.title = it },
            textStyle = TextStyle(fontSize = 14.sp, fontWeight = FontWeight.Medium,
color = Color(0xFF334155)),
            modifier = Modifier.weight(1f)
        )
    }
}

// 内部嵌套子内容的展开动画（核心：消除套娃感，利用缩进展示层次）
AnimatedVisibility(visible = isExpanded) {
    Column(
        modifier = Modifier
            .padding(start = 24.dp, top = 8.dp) // 关键：左侧大幅缩进 24.dp
            .fillMaxWidth()
    ) {
        block.childBlocks.forEach { childBlock ->
            when (childBlock) {
                is NoteBlock.Text -> TextBlockComponent(childBlock)
                is NoteBlock.Image -> ImageBlockComponent(childBlock)
                is NoteBlock.Formula -> FormulaBlockComponent(childBlock)
                else -> {}
            }
            Spacer(modifier = Modifier.height(6.dp))
        }
    }
}
}
}
}

```